



Systemes Numériques / FPGA

SÉBASTIEN VAN
CAUWENBERGHE
(S3V@ECAM.BE)

Introduction

FPGA et Zynq:

- Process en VHDL
- Logique Séquentielle / Synchrone
- FSM
- Demo

TEACH A COURSE



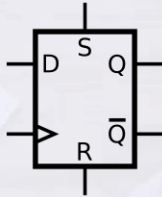
Cours 1 : Récap

- Logique combinatoire et comment créer des circuits simples
- Rappel d'arithmétique
- Outils
- Premiers exercices

Design Synchrone

- Design Asynchrone : Design logique dont les éléments changent indépendamment d'une référence commune
- Design Combinatoire : Design dont la sortie ne dépend que des entrées
- Design Séquentiel : Design dont la sortie dépend des entrées et de l'état précédent du design
- Design Synchrone : Design dont tous les éléments changent sur une référence commune
- Horloge : Signal de référence généré par un oscillateur
- Reset : Signal qui remet le système dans un état connu (peut être conditionné par l'horloge (synchrone) ou pas (asynchrone))
- See *Introduction to Logic Circuits & Logic Design with VHDL* book chapter 7.

D Flip-flop

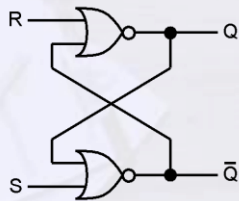


- > Clock In
- S Set
- R Reset
- D Data In
- Q Data Out
- /Q Not Data Out
- CE Clock Enable

Logic Table

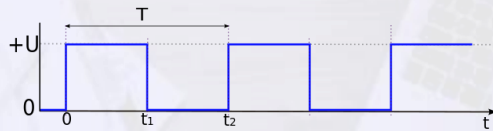
CLR	Inputs			Outputs	
	CE	D	C	Q	
1	X	X	X	0	
0	0	X	X	No Change	
0	1	D	1	D	

D Flip-flop Issues

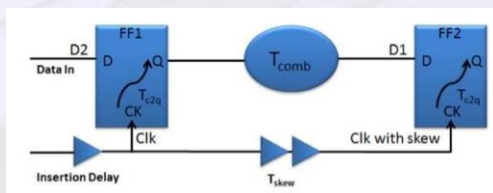


- Coming from RS Latch
- Feedback loop can have oscillatory behavior
- If data is changed

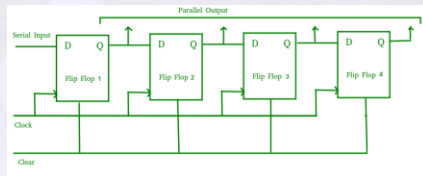
Sequential design



- Basé sur un signal d'horloge de période T
- F est une fonction combinatoire



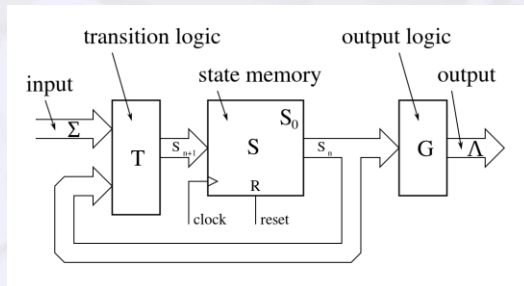
Sequential design – Shift register



- Simplest possible circuit with D Flip-Flops.

- Explain how this works :)

Sequential design – Finite State machines formalism

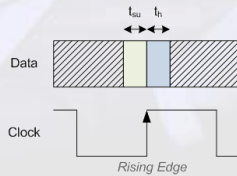
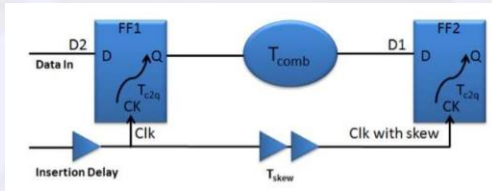


- $D_{t+1} \leq f(Q_t)$ (Moore state machine)
- $D_t = f(Q_{t-1}, D_t)$ (Mealy state machine)
- D is the next state
- F is the transition function
- There can be an output function (G) placed after Q
- D is the DFF input and Q the DFF output

Sequential design – How to

- Determine what function should be done
- Define a state machine (Moore/Mealy) with the accompanying states (One-hot, Binary, Gray)
- Define the transitions between the states
- Define the mapping between the states encoding and the output
- Example 1: Design a traffic light
- Example 2: Explain how an AMBA APB master works
(<https://developer.arm.com/documentation/ih0024/latest/>)

Sequential design



- Limité par T_{CLK}
- T_{setup} : Temps où la donnée doit être stable avant le flanc d'horloge
- T_{hold} : Temps où la donnée doit rester stable après le flanc d'horloge
- T_{c2q} : Temps entre la flanc d'horloge et la mise à jour de la sortie
- T_{skew} : Délai de propagation de

l'horloge entre 2 flip-flops

- T_{logic} : Délai de propagation dans la logique (routage et fonction combinatoire)

$$T_{c2q} + T_{logic} + T_{setup} \leq T_{clk} + T_{skew}$$

$$T_{c2q} + T_{logic} \geq T_{hold} + T_{skew}$$

How to design logic ?

- Choisir ses contraintes : Vitesse (Fréquence), Latence, Surface (-> Conso)
- La vitesse dépend du design et la physique (Process, Tension, Température)
- Est-ce que tout est faisable en un cycle ?
- Chercher les blocs de base (Compteur, FSM, Opérateurs arithmétique, Opérateurs logique, comparaisons, registres a décalage)
- Dessiner le schéma d'architecture
- Calculer la taille des registres (optionnel)
- Coder en VHDL
- Vérifier avec un testbench
- Synthétiser
- Uploader sur la carte
- Prier
- Débugger
- Profit :)

How to Verify logic ?

- Simplify your life, use Vunit/OSVVM, UVM
- For small sizes, test all combinations
- For medium/large sizes, test edge cases, randomize other values.
- Combo 2^n test cases
- With sequential stuff gets complicated
- Test all states is impossible
- Test States transition
- Randomize and compare with a model
- Use Verification Components/Bus functional Models (For UART, SPI, I2C, AXI)

Different kind of Synchronous designs

- General case => Finite state machine
- Counters
- Data Path
- Stream
- Bus

Sequential logic exercises

Exercise time:

- Use Digital again
- Solve the exercises on the lab2 sheet

